

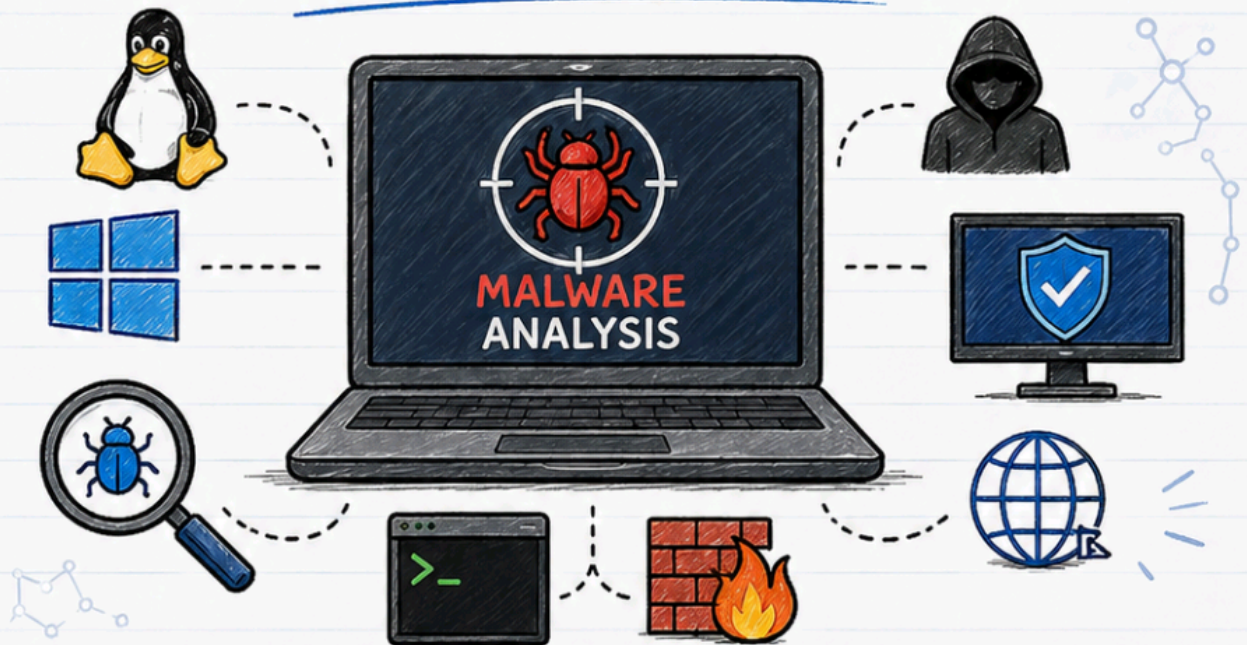


BLUETEAMERS
— DEFEND. DETECT. RESPOND. —

OPEN SOURCE MALWARE ANALYSIS HOME LAB

Beginner Friendly Practical Guide

Build Your Own Malware Analysis Lab Using Free Tools



STEP-BY-STEP
GUIDE



REAL
SCREENSHOTS



PRACTICAL
EXERCISES



100%
OPEN SOURCE



Prepared by Blueteamers



@blueteamers_



WHAT IS MALWARE ANALYSIS?



HOOK:

How do security analysts safely investigate malware without infecting their own computer?

DEFINITION:

Malware Analysis is the process of studying **malicious files** to understand what they do, how they behave and how to detect or stop them.

TYPES OF MALWARE ANALYSIS:

1. STATIC ANALYSIS

- Analyze the file without executing it.
- Look at its code, structure, strings, headers, imports, etc.



2. DYNAMIC ANALYSIS

- Execute the file in a safe environment.
- Observe its behavior, network activity, and system changes.



REAL WORLD EXAMPLE:



Employee downloads fake **invoice.pdf.exe**



Computer gets infected



SOC Analyst investigates malware



Creates detection rules (YARA / SIEM)



Attack is detected and blocked

TOOLS USED:



Static Tools



Dynamic Tools



Network Tools



Memory Analysis

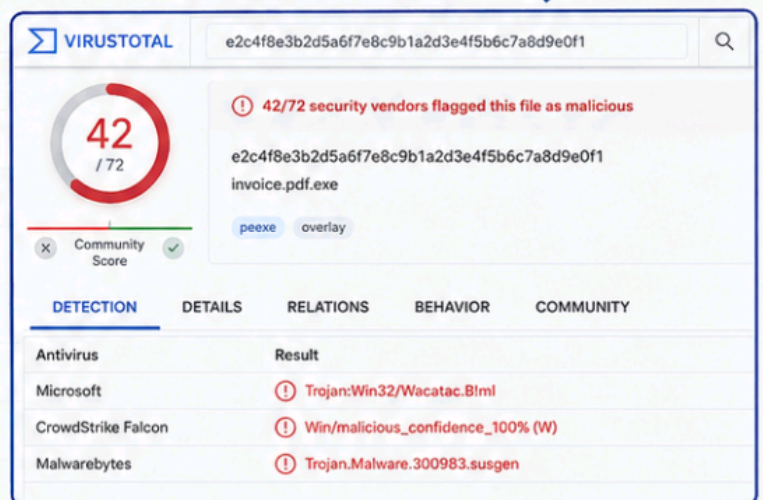


YARA Rules

WHY IT MATTERS?

Malware analysis helps SOC Analysts find **Indicators of Compromise (IOCs)**, create detection rules and protect organizations from future attacks.

REAL SCREENSHOT:



VIRUSTOTAL e2c4f8e3b2d5a6f7e8c9b1a2d3e4f5b6c7a8d9e0f1

42 / 72

42/72 security vendors flagged this file as malicious

e2c4f8e3b2d5a6f7e8c9b1a2d3e4f5b6c7a8d9e0f1
invoice.pdf.exe

peexe overlay

Community Score

DETECTION	DETAILS	RELATIONS	BEHAVIOR	COMMUNITY
Antivirus	Result			
Microsoft	Trojan:Win32/Wacatac.B!ml			
CrowdStrike Falcon	Win/malicious_confidence_100% (W)			
Malwarebytes	Trojan.Malware.300983.susgen			



TYPES OF MALWARE ANALYSIS

There are two main ways to analyze malware.

1. STATIC ANALYSIS



- ✓ Analyze the file without executing it.
- ✓ Safe and fast.
- ✓ Finds code, strings, imports, and structure.
- ✓ Helps to extract IOCs.

Goal: Understand what the malware is capable of doing.

2. DYNAMIC ANALYSIS

- ✓ Execute the file in a safe environment (sandbox/VM).
- ✓ Observe real behavior.
- ✓ Monitor system changes, network traffic, persistence, etc.
- ✓ Risky without isolation.



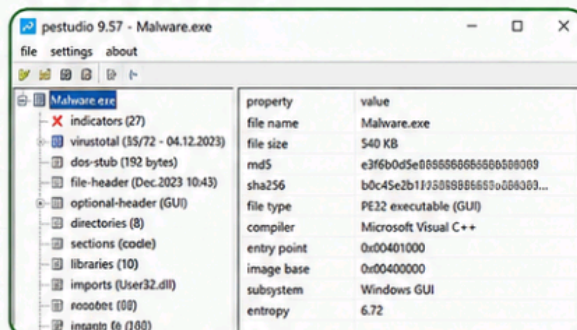
Goal: Understand what the malware actually does.

STATIC vs DYNAMIC

Feature	Static Analysis	Dynamic Analysis
Execution	Not Executed	Executed in Sandbox / VM
Safety	Very Safe	Risky (Needs Isolation)
Speed	Fast	Slow
Information	Code, Strings, Metadata	Behavior, Network, Changes
Tools	PEStudio, Strings, DIE	Procmon, Wireshark, Sandbox
Best For	Initial Inspection	Deep Behavior Analysis

STATIC TOOL EXAMPLE:

PESTUDIO (Static Analysis)



DYNAMIC TOOL EXAMPLE:

ANY.RUN SANDBOX (Dynamic Analysis)



KEY TAKEAWAY

- Use **STATIC** analysis first (safe & fast).
- Use **DYNAMIC** analysis to discover real behavior.
- Combine both to get **full visibility**.



TIP

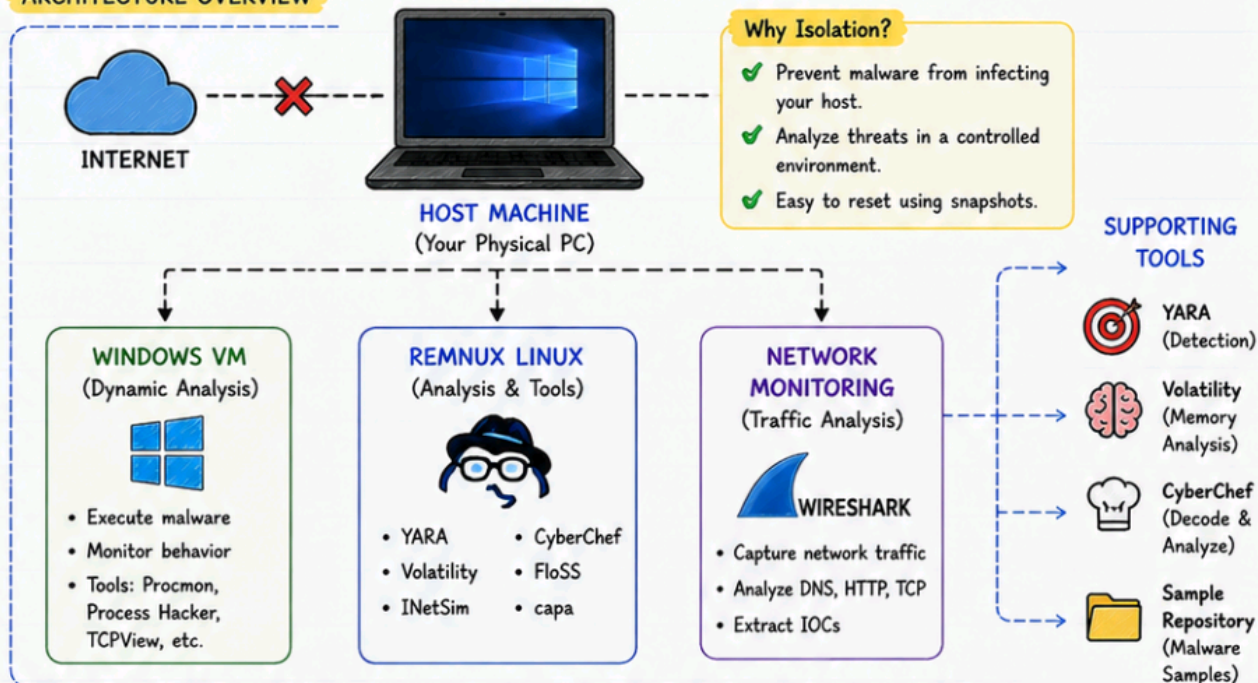
Always use a **isolated environment** (VM / Sandbox) and take **snapshots** before executing any suspicious file.



HOME LAB ARCHITECTURE

A good lab setup keeps you safe, organized and gives you all the tools needed for malware analysis.

ARCHITECTURE OVERVIEW



COMPONENTS EXPLAINED

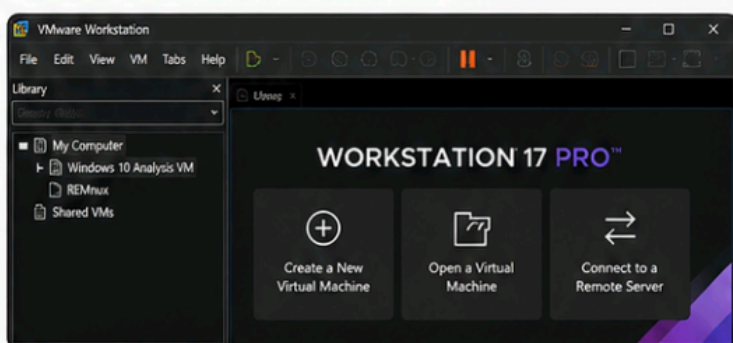
- **Host Machine:** Your main computer. Runs virtualization software (VMware/VirtualBox).
- **Windows VM:** Used to run malware and observe real-time behavior.
- **REMnux Linux:** Full of pre-installed malware analysis tools.
- **Network Monitoring:** Captures and analyzes malware network traffic.
- **Supporting Tools:** YARA, Volatility, CyberChef and sample repository for deeper analysis.

TIP

Always take a **SNAPSHOT** before running any malware. This allows you to revert the VM to a clean state instantly.



LAB SETUP EXAMPLE (Using VMware)



GOOD PRACTICES

- ✓ Keep VMs disconnected from the internet (use Host-Only or NAT).
- ✓ Use snapshots before executing any suspicious file.
- ✓ Regularly update your tools.
- ✓ Store malware samples in a dedicated folder.



Prepared by Blueteamers



@blueteamers_





INSTALL VMWARE / VIRTUALBOX

Virtualization lets us run multiple OS safely on our host machine.

WHY VIRTUALIZATION?

- ✓ Run malware safely in isolated VMs.
- ✓ Easy to snapshot and revert.
- ✓ Test, learn and practice without risk.
- ✓ Save time, money and hardware.

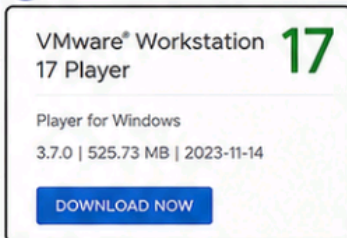


OR



STEPS: INSTALL & CREATE VM (EXAMPLE: VMWARE)

1 Download VMware



Download VMware Workstation Player from the official website.

2 Install VMware



Run the installer and follow the setup wizard.

3 Create a New VM



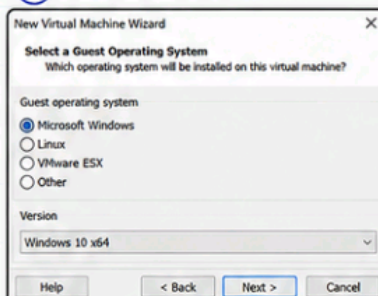
Click on "Create a New Virtual Machine".

4 Choose Installer Disc



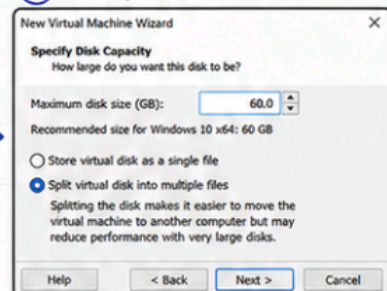
Select "Typical (recommended)" and click Next.

5 Guest OS Selection



Choose Microsoft Windows and version (e.g., Windows 10 x64).

6 Configure Hardware



Allocate disk space (e.g., 60 GB) and click Next.

TIPS

- Allocate at least 2 CPUs and 4 GB RAM for smooth performance.
- Use NAT Network for internet access in VM.
- Always take a **SNAPSHOT** after installing the OS and tools.



RECOMMENDED VM SPECS (BEGINNER)

Resource	Recommended
CPU	2 Cores
RAM	4 - 8 GB
Disk	60 - 100 GB
Network	NAT (For internet)
Snapshot	Always after setup



INSTALL WINDOWS ANALYSIS VM

We will use Windows 10 (or Windows 11) as our analysis machine and install essential malware analysis tools.

REQUIREMENTS:

- ✓ Windows 10 (64-bit) ISO
- ✓ VM with 2 CPU, 4-8 GB RAM, 60 GB Disk
- ✓ Internet (only during installation)
- ✓ Take a snapshot after setup
- ✓ Keep the VM isolated (Host-Only / NAT)

INSTALLATION STEPS (SUMMARY):

- ① Create VM (done in previous page)
- ② Mount Windows ISO
- ③ Install Windows
- ④ Set Administrator account
- ⑤ Install VMware Tools / Guest Additions
- ⑥ Disable Windows Update (optional)
- ⑦ Take a clean **SNAPSHOT**

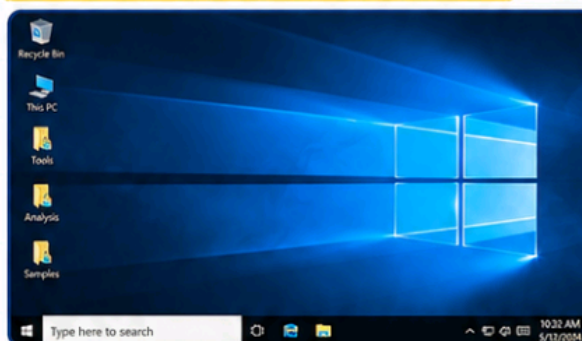


Windows
10 / 11

VM CONFIGURATION (EXAMPLE):

Setting	Value
Name	Win10-Analysis
OS Type	Microsoft Windows 10 (64-bit)
CPU	2 Cores
RAM	4 - 8 GB
Disk	60 GB (Thin Provisioned)
Network	Host-Only / NAT
Snapshot	After clean installation

REAL SCREENSHOT: WINDOWS VM DESKTOP



NOTE:

Keep this VM clean and dedicated only for analysis.

ESSENTIAL TOOLS TO INSTALL



Sysinternals
(Microsoft)



Process
Hacker



Procmon
(Process Monitor)



PEStudio
(Static Analysis)



Detect It Easy
(DIE)

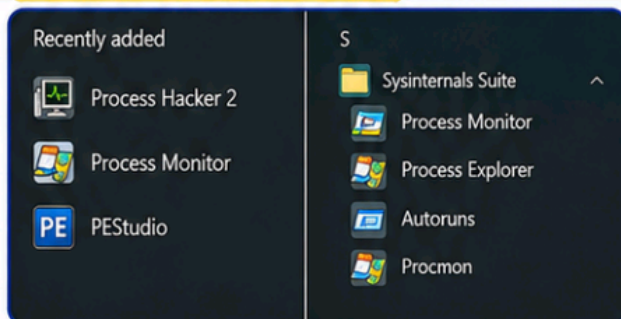


7-Zip
(File Extractor)



Notepad++
(Text Viewer)

SCREENSHOT: TOOLS INSTALLED



TIPS

- Always keep the VM off the internet when analyzing malware.
- Use Host-Only network for extra safety.
- Take a **SNAPSHOT** before executing any suspicious file.



TIP

After installation, disable Windows Defender real-time protection only during analysis (enable it again after you finish).



INSTALL REMnux LINUX

REMnux is a Linux distribution pre-loaded with malware analysis and reverse engineering tools.

ABOUT REMNUX

- ✓ Based on Ubuntu LTS.
- ✓ Designed for malware analysis, reverse engineering and DFIR.
- ✓ Comes with 300+ tools out of the box.
- ✓ Regularly updated and community supported.
- ✓ Works perfectly in Virtual Machines.

KEY FEATURES

-  Pre-installed Analysis Tools
YARA, Volatility, CyberChef, Ghidra, radare2, etc.
-  Malware Analysis Ready
Static, Dynamic, Network and Memory Analysis.
-  Safe & Isolated
Run malware without risking your host.
-  Open Source & Free
100% open source and community driven.

INSTALLATION STEPS (SUMMARY)

- ① Download REMnux ISO from official website.
- ② Create a New VM (Linux).
- ③ Mount REMnux ISO to VM.
- ④ Start the VM and boot from ISO.
- ⑤ Follow the installer.
- ⑥ Configure user and password.
- ⑦ Complete installation & reboot.
- ⑧ Update the system.

REAL SCREENSHOT: REMNUX DESKTOP



TOOLS AVAILABLE IN REMNUX (EXAMPLES)

YARA

YARA
Pattern matching
& detection



Volatility
Memory forensics
framework



CyberChef
Decode, Encode
& Analyze data



Ghidra
Reverse
Engineering



radare2
Binary analysis
framework



Floss
Extract strings
from binaries



INetSim
Network services
simulator



Wireshark
Network protocol
analyzer



Binwalk
Firmware analysis
tool



Capa
Detect capabilities
in malware



Mobsf
Mobile security
framework



PE
PEStudio
PE file analysis
(also on Windows)



TIP

Allocate at least 4 CPU cores and 8 GB RAM to REMnux VM for smooth performance. Keep the VM isolated (Host-Only).

NOTE

After installation, run the following commands:

```
sudo apt update && sudo apt upgrade -y
sudo remnux upgrade
```



Prepared by Blueteamers



@blueteamers_



ANALYZE MALWARE NETWORK TRAFFIC

Malware often communicates with remote servers to steal data, download payloads or send system information.

We capture and analyze this traffic using Wireshark.

WHY NETWORK ANALYSIS?

- ✓ Discover malicious IP addresses & domains.
- ✓ Detect C2 (Command & Control) communication.
- ✓ Identify data exfiltration.
- ✓ Understand malware behavior in real-time.



WHAT TO LOOK FOR?



DNS Requests
Look for suspicious domain resolutions.



HTTP Traffic
Check for data uploads, downloads and beacons.

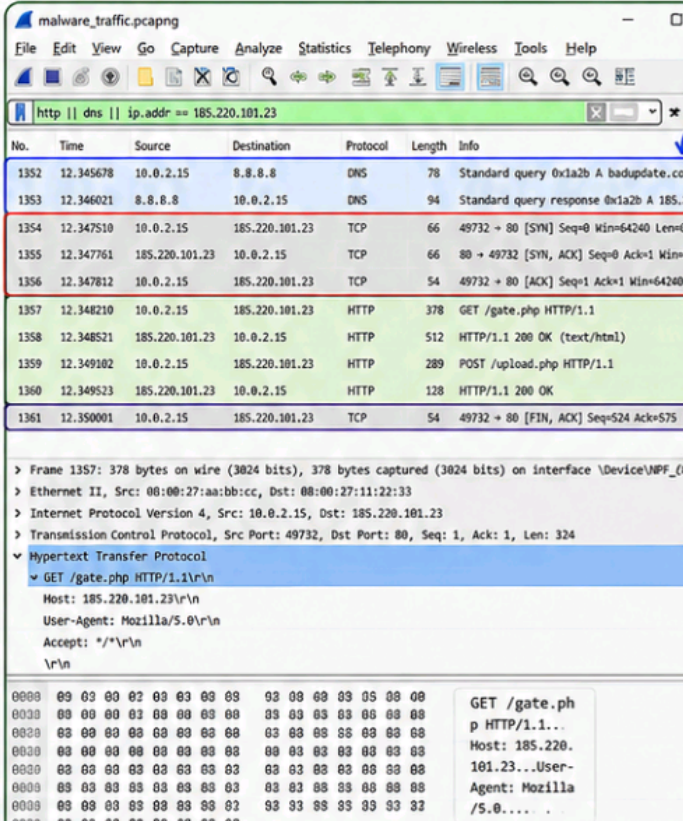


TCP Connections
Identify unusual external connections.



Suspicious IPs
Check for unknown or malicious IP addresses.

REAL SCREENSHOT: WIRESHARK TRAFFIC CAPTURE



The screenshot shows a Wireshark capture of network traffic. The packet list on the left shows several packets. The packet details pane on the right shows the selected packet (No. 1357) which is an HTTP GET request for /gate.php. The packet bytes pane at the bottom shows the raw data of the HTTP request.

No.	Time	Source	Destination	Protocol	Length	Info
1352	12.345678	10.0.2.15	8.8.8.8	DNS	78	Standard query 0x1a2b A badupdate.com
1353	12.346021	8.8.8.8	10.0.2.15	DNS	94	Standard query response 0x1a2b A 185.220.101.23
1354	12.347510	10.0.2.15	185.220.101.23	TCP	66	49732 → 80 [SYN] Seq=0 Win=64240 Len=0
1355	12.347761	185.220.101.23	10.0.2.15	TCP	66	80 → 49732 [SYN, ACK] Seq=0 Ack=1 Win=64240
1356	12.347812	10.0.2.15	185.220.101.23	TCP	54	49732 → 80 [ACK] Seq=1 Ack=1 Win=64240
1357	12.348210	10.0.2.15	185.220.101.23	HTTP	378	GET /gate.php HTTP/1.1
1358	12.348521	185.220.101.23	10.0.2.15	HTTP	512	HTTP/1.1 200 OK (text/html)
1359	12.349102	10.0.2.15	185.220.101.23	HTTP	289	POST /upload.php HTTP/1.1
1360	12.349523	185.220.101.23	10.0.2.15	HTTP	128	HTTP/1.1 200 OK
1361	12.350001	10.0.2.15	185.220.101.23	TCP	54	49732 → 80 [FIN, ACK] Seq=524 Ack=575

Annotations on the screenshot:

- DNS Query for badupdate.com**: Points to packet 1352.
- Connection to Malicious IP**: Points to packet 1354.
- HTTP Request (Possible Beacon)**: Points to packet 1357.
- Data Sent to Server**: Points to packet 1359.
- HTTP Response from Server**: Points to packet 1360.

USEFUL WIRESHARK FILTERS

- `dns` → Show DNS traffic
- `http` → Show HTTP traffic
- `tcp` → Show TCP traffic
- `udp` → Show UDP traffic
- `ip.addr == 185.220.101.23` → Filter by IP address
- `dns.qry.name contains ".com"` → Track domain queries
- `http.request.method == POST` → Show POST requests



KEY TAKEAWAY

- Always capture traffic while malware is running.
- Focus on **DNS**, **HTTP**, **TCP** and suspicious IPs.
- Extract **IOCs** (IP, Domain, URL) for further analysis and detection.



TIP

Use FakeNet-NG or INetSim in REMnux to simulate internet services and safely observe malware behavior.





STATIC ANALYSIS WORKFLOW

Static analysis helps us understand what a file is capable of without executing it.

STATIC ANALYSIS FLOW



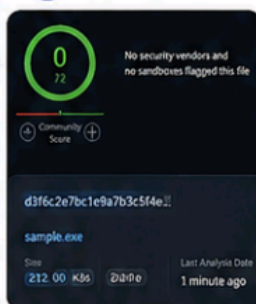
WHAT WE LEARN FROM STATIC ANALYSIS?

- ✓ File Type & Architecture (32-bit / 64-bit)
- ✓ Hash Values (MD5, SHA1, SHA256)
- ✓ Imported Libraries & Functions
- ✓ Sections, Entropy, Compiler Info
- ✓ Suspicious Strings & Artifacts
- ✓ Possible Capabilities & Behaviors

KEY POINT

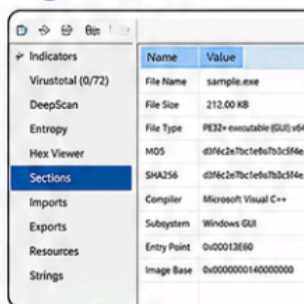
Static analysis is safe, fast and gives you important insights before running the malware in a lab.

① VIRUSTOTAL



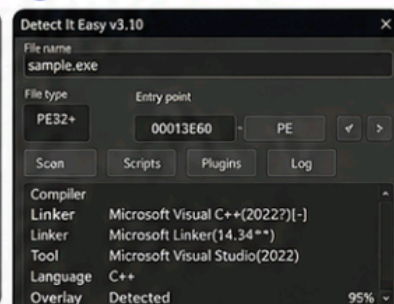
- Check reputation of file.
- View detections from AV engines.
- Extract basic file information.

② PESTUDIO



- Detailed PE file information.
- Sections, Imports, Exports.
- Detect anomalies and red flags.

③ DETECT IT EASY (DIE)



- Guess compiler, linker and packer.
- Detect packers or protectors.
- Helps in understanding the build.

④ STRINGS (EXTRACT ARTIFACTS)

```

remnux@remnux:~/malware$ strings sample.exe | less
http://malicious-domain.com/gate.php
/api/v1/upload
powershell.exe -nop -w hidden -enc
cmd.exe /c whoami
C:\Users\Public\svchost.exe
MutexGlobal\{3A17B8C4-2F63-4B22-A1E7}
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
%APPDATA%\Microsoft\svchost.exe
  
```

WHAT TO EXTRACT?

- 🌐 Domains / URLs
- 📍 IP Addresses
- 📁 File Paths
- 📦 Registry Keys
- 🔒 Mutex Names
- 👤 User Agents
- 📄 Commands
- ✉ Email Addresses

TIPS

- Always work on a copy of the sample.
- Never upload sensitive data to public sites.
- Take a **SNAPSHOT** before static analysis.
- Note down all **IOCs** for reporting.



NOTE

Static analysis gives you a map of the malware. It tells you what the file **CAN** do. Next step: Let's see what it **DOES** in a live environment!





WATCH MALWARE BEHAVIOR (DYNAMIC ANALYSIS)

Dynamic analysis helps us observe how malware behaves when it runs in a safe and isolated environment.

WHY DYNAMIC ANALYSIS?

- ✓ Observe real-time behavior.
- ✓ Identify malicious activities.
- ✓ Detect persistence mechanisms.
- ✓ Monitor network connections.
- ✓ See what static analysis cannot find.



WHAT WE MONITOR?



PROCESSES

New or suspicious processes



REGISTRY

Changes to registry keys & values



FILES

Files created, modified or deleted



STARTUP

Persistence via startup entries



NETWORK

Connections to remote servers

DYNAMIC ANALYSIS WORKFLOW



Run Malware in VM



Observe System Changes



Monitor Network Activity



Identify Malicious Behavior



Document & Extract IOCs

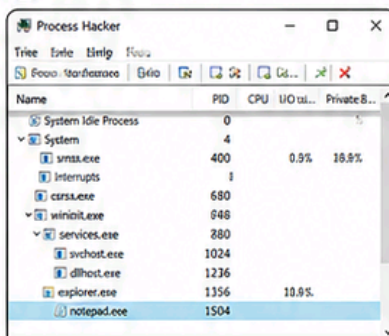
IMPORTANT!



- Always use a clean VM (Snapshot).
- Never run malware on your host machine.
- Disconnect internet if not needed.
- Revert to snapshot after analysis.

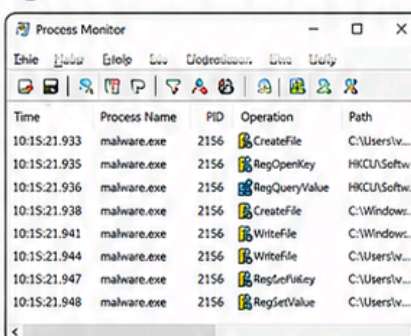
REAL TOOLS WE USE

1 PROCESS HACKER



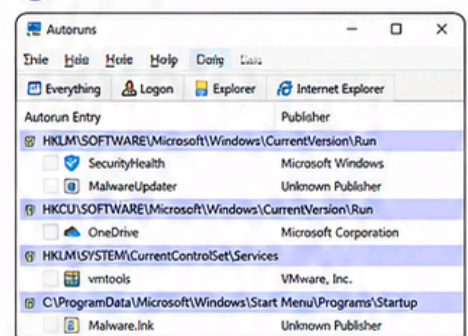
- View running processes.
- Detect suspicious or unknown processes.
- Check CPU, memory and handles.

2 PROCmon (PROCESS MONITOR)



- Monitor file, registry and process activity.
- See what changes malware makes.
- Powerful for spotting persistence.

3 AUTORUNS



- Detect startup programs.
- Find persistence mechanisms.
- Check scheduled tasks & services.

BEHAVIOR EXAMPLE

MALICIOUS BEHAVIOR

- ✓ Creates files in AppData/Temp → C:\Users\...\AppData\mal.exe
- ✓ Modifies registry for persistence → HKCU\Software\...\Run\mal
- ✓ Adds startup entries → New startup entry added
- ✓ Connects to suspicious IP → 185.220.101.23:80 (TCP)
- ✓ Downloads additional payloads → GET /gate.php (HTTP)

INDICATOR FOUND

PRO TIP

- Observe patiently and don't rush.
- Correlate process, registry, file and network events.
- Take notes and screenshots.
- Extract IOCs for reporting.



GOAL: UNDERSTAND THE STORY BEHIND THE MALWARE

Dynamic analysis shows us the full attack path and attacker's intentions.





MEMORY FORENSICS WITH VOLATILITY

Memory (RAM) contains valuable information that disappears when a system is powered off. Volatility helps us extract it!

WHY MEMORY ANALYSIS?

- ✓ Find running malware & hidden processes.
- ✓ Detect code injection & malicious DLLs.
- ✓ Recover network connections.
- ✓ Extract credentials & encryption keys.
- ✓ Carve files & artifacts.



WHAT MEMORY CONTAINS?

PROCESSES	NETWORK	REGISTRY	FILES	CREDENTIALS
Running & hidden processes	Connections, ports, IPs	Keys, values, changes	Open files, file paths	Passwords, tokens, keys

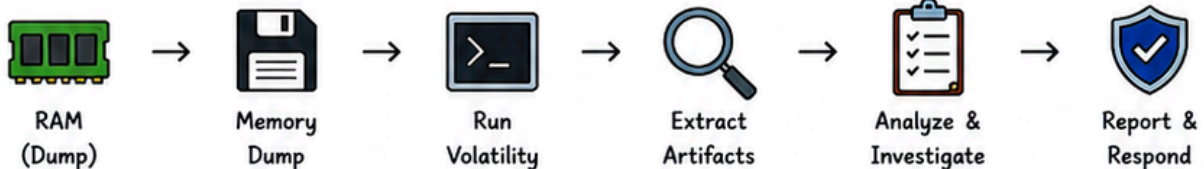
USEFUL VOLATILITY COMMANDS

COMMAND	PURPOSE
windows.info	Get OS, profile & basic info
pslist	List running processes
pstree	Show process tree
netscan	List network connections
dlllist	List loaded DLLs
malfind	Find injected code & malware
filesfan	Scan for file objects
handles	List open handles
cmdline	Show process command lines
hashdump	Dump password hashes

REAL SCREENSHOT (VOLATILITY OUTPUT)

```
remnux@remnux:~/memdump$ volatility -f memdump.raw windows.pslist
Volatility 3 Framework 2.4.1
PID  PPID  ImageFileName      Offset(V)
-----
4      0      System             0xffffffff800`02a06020
360    4      Registry           0xffffffff800`02c60020
476    4      smss.exe           0xffffffff800`02d22020
604    588    csrss.exe          0xffffffff800`02eae020
660    588    wininit.exe        0xffffffff800`02f1e020
716    656    services.exe       0xffffffff800`02f73020
728    656    lsass.exe          0xffffffff800`02fd3020
916    716    svchost.exe        0xffffffff800`0303b020
1024   716    svchost.exe        0xffffffff800`0318a020
1220   716    svchost.exe        0xffffffff800`031f6020
1356   716    explorer.exe       0xffffffff800`0328a020
1504   1356   notepad.exe        0xffffffff800`033f6020
1720   716    cmd.exe            0xffffffff800`034b2020
remnux@remnux:~/memdump$
```

MEMORY ANALYSIS WORKFLOW



TIPS

- Always capture memory before shutting down the system.
- Use the correct profile (windows.info).
- Combine memory analysis with other artifacts.
- Memory never lies, attackers can't hide everything!



NOTE

Memory analysis is one of the most powerful techniques in DFIR and SOC investigations. It can reveal what other tools cannot see!



GOAL: EXTRACT HIDDEN TRUTH FROM MEMORY

Memory forensics helps us uncover the attacker's activities even when the system looks normal.

EXTRACT INDICATORS OF COMPROMISE (IOCs)

IOCs are pieces of evidence that help us detect, block and respond to cyber threats.

WHAT ARE IOCs?

IOCs are artifacts or observable elements found during analysis that indicate malicious activity or an attack.



TYPES OF IOCs

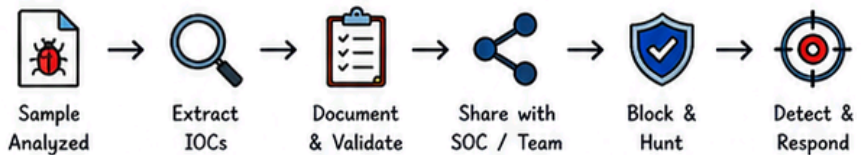
MD5 / SHA256	IP ADDRESS	DOMAIN	URL	REGISTRY	MUTEX	FILE PATH
File Hashes	Malicious IPs	Malicious Domains	Malicious URLs	Malicious Registry Keys	Mutex Names	Suspicious File Paths

IOC EXTRACTION SOURCES

- ✓ Static Analytcs Tools (PEStudio, DIE, Strings)
- ✓ Dynamic Analysis Tools (Procmon, Wireshark, etc.)
- ✓ Memory Analysis (Volatility, etc.)
- ✓ Threat Intelligence Feeds (VirusTotal, MISP, etc.)
- ✓ Malware Sandbox Reports



IOC EXTRACTION WORKFLOW



REAL EXAMPLE: IOC REPORT

IOC REPORT - Sample: malware.exe

TYPE	VALUE	SOURCE	DESCRIPTION
SHA256	d3f6c2e7bc1e9a7b3c5f4e1a...	Static Analysis	Malware File Hash
MD5	8e2d3c7f1a4b5c6d7e8f9a0b	Static Analysis	MD5 Hash
IP	185.220.101.23	Wireshark	C2 Server IP
DOMAIN	badupdate.com	Wireshark	Malicious Domain
URL	hxxp://badupdate.com/gate.php	Wireshark	Malicious URL
REGISTRY	HKCU\Software\mal\run	Procmon	Persistence Key
FILE PATH	C:\Users\Public\svchost.exe	Procmon	Dropped File
MUTEX	Global\mymutex123	Procmon	Mutex Name

WHY IOCS ARE IMPORTANT?

- ✓ Help detect current and future attacks.
- ✓ Enable threat hunting & correlation.
- ✓ Used for blocking and preventing threats.
- ✓ Build strong threat intelligence.
- ✓ Improve incident response speed.



COMMON IOC COLLECTION COMMANDS

- `> strings malware.exe | more`
→ Extract strings
- `> Get-FileHash malware.exe -Algorithm SHA256`
→ Get File Hash
- `Wireshark (Filter: ip.addr==185.220.101.23)`
→ Capture Network IOCs
- `reg query HKCU\Software\mal /s`
→ Find Registry IOCs

IOC REPORT TEMPLATE

	TYPE	VALUE	SOURCE	CONFIDENCE	NOTES
	SHA256	xxxxxxux...	Static	High	Malware File
	IP	x.x.x.x	Wireshark	High	C2 Server
	DOMAIN	example.com	Wireshark	Medium	Malicious Domain
	URL	hxxp://...	Wireshark	Medium	Malicious URL
	REGISTRY	HKCU\...	Procmon	High	Persistence
	FILE PATH	C:\Users\...	Procmon	High	Dropped File
	MUTEX	Global\...	Procmon	Medium	Mutex Name

TIP



Always validate IOCs before using them.
Not every suspicious item is real!

NOTE

Good IOC reporting = Better detection,
faster response and stronger defense.





WRITE YOUR FIRST YARA RULE

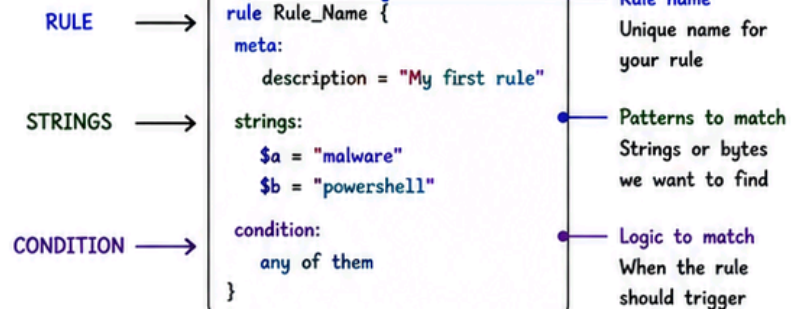
YARA is a powerful tool that helps us identify and classify malware based on patterns or strings.

YARA RULE BASICS

- ✓ Rule → Rule name
- ✓ Strings → Patterns to look for
- ✓ Condition → When rule should match



YARA RULE STRUCTURE



SIMPLE YARA RULE EXAMPLE

```

rule Detect_Malware
{
  meta:
    description = "Detects common malware strings"
    author = "BlueTeamers"

  strings:
    $a = "powershell" nocase
    $b = "cmd.exe" nocase
    $c = "http://"

  condition:
    any of them
}

```

EXPLANATION

- `rule Detect_Malware` → Rule name
- `meta` → Information about the rule
- `strings` → Patterns we are searching for
 - `$a` and `$b` are case-insensitive
 - `$c` is case-sensitive
- `condition` → Rule will match if any string is found



CREATE AND TEST YOUR RULE

① CREATE RULE FILE

```

malware.yar x
1 rule Detect_Malware
2 {
3   strings:
4     $a = "powershell"
5     $b = "cmd.exe"
6   condition:
7     any of them
8 }

```

Save as: malware.yar

② SCAN A FILE

```

C:\>
C:\YARA>yara malware.yar
sample.exe

```

Run YARA against a file or folder

③ RULE MATCHED

```

C:\>
Detect_Malware sample.exe
$a = "powershell"
$b = "cmd.exe"
0x00102020
0x00102038

```

If patterns are found, rule will match!

④ NO MATCH

```

C:\>
C:\YARA>yara malware.yar
clean_file.exe

No matches.

```

If no pattern found, no match.

TIPS

- Start with simple strings.
- Use `nocase` for case-insensitive matching.
- Test your rules with different samples.
- Add more strings to improve accuracy.
- Keep your rules clean and easy to read.



NOTE

- YARA rules are used in SOC, DFIR and Threat Hunting.
- They help automate malware detection.
- Your rule, your detection power 💪





SCAN FILES WITH YARA

Once you have a **YARA rule**, you can use it to scan files or folders to detect matching patterns.

YARA SCAN WORKFLOW



BASIC YARA SYNTAX (RECAP)

```

rule Detect_Malware
{
  strings:
    $a = "powershell" nocase
    $b = "cmd.exe" nocase

  condition:
    any of them
}
  
```

COMMON YARA COMMANDS

COMMAND	DESCRIPTION
yara rule.yar file.exe	Scan a file with a rule
yara rule.yar folder/	Scan all files in a folder
yara -r rule.yar folder/	Scan folder recursively
yara -s rule.yar file.exe	Show matching strings
yara -w rule.yar file.exe	Print full match details

REAL EXAMPLE: SCAN FILE

```

remnux@remnux:~/yara$ yara malware.yar sample.exe
Detect_Malware sample.exe
  $a = "powershell"
  $b = "cmd.exe"
  0x1a2b3c = "powershell"
  0x1a2b50 = "cmd.exe"
remnux@remnux:~/yara$
  
```

✓ Match Found! Rule Detect_Malware matched the file.

REAL EXAMPLE: SCAN FOLDER

```

remnux@remnux:~/yara$ yara -r malware.yar samples/
Detect_Malware samples/mal1.exe
Detect_Malware samples/mal2.dll
Detect_Malware samples/suspicious.bin
3 files scanned, 3 matches.
  
```

✓ Multiple files matched in the folder.

SHOW STRINGS MATCHED

```

remnux@remnux:~/yara$ yara -s malware.yar sample.exe
Detect_Malware sample.exe
  $a = "powershell"
  $b = "cmd.exe"
  $c = "http://"
  0x1a2b3c = "powershell"
  0x1a2b50 = "cmd.exe"
  0x1a2b80 = "http://malicious.com"
  
```

TIPS FOR EFFECTIVE SCANNING

- ✓ Use recursive scan (-r) for entire folders.
- ✓ Keep your YARA rules updated.
- ✓ Combine multiple rules for better detection.
- ✓ Use -s to see what matched.
- ✓ Verify matches before taking action.



NOTE



- Not every match is a threat.
- Always investigate the context.
- False positives can happen.
- Use YARA as a detection, not as a final verdict.

EXAMPLE SCAN SCENARIOS

- Scan downloads folder for malware.
- Scan email attachments.
- Hunt for indicators across the endpoint.
- Automate scans with scripts or SIEM.



GOAL: AUTOMATE DETECTION AND FIND THREATS FASTER

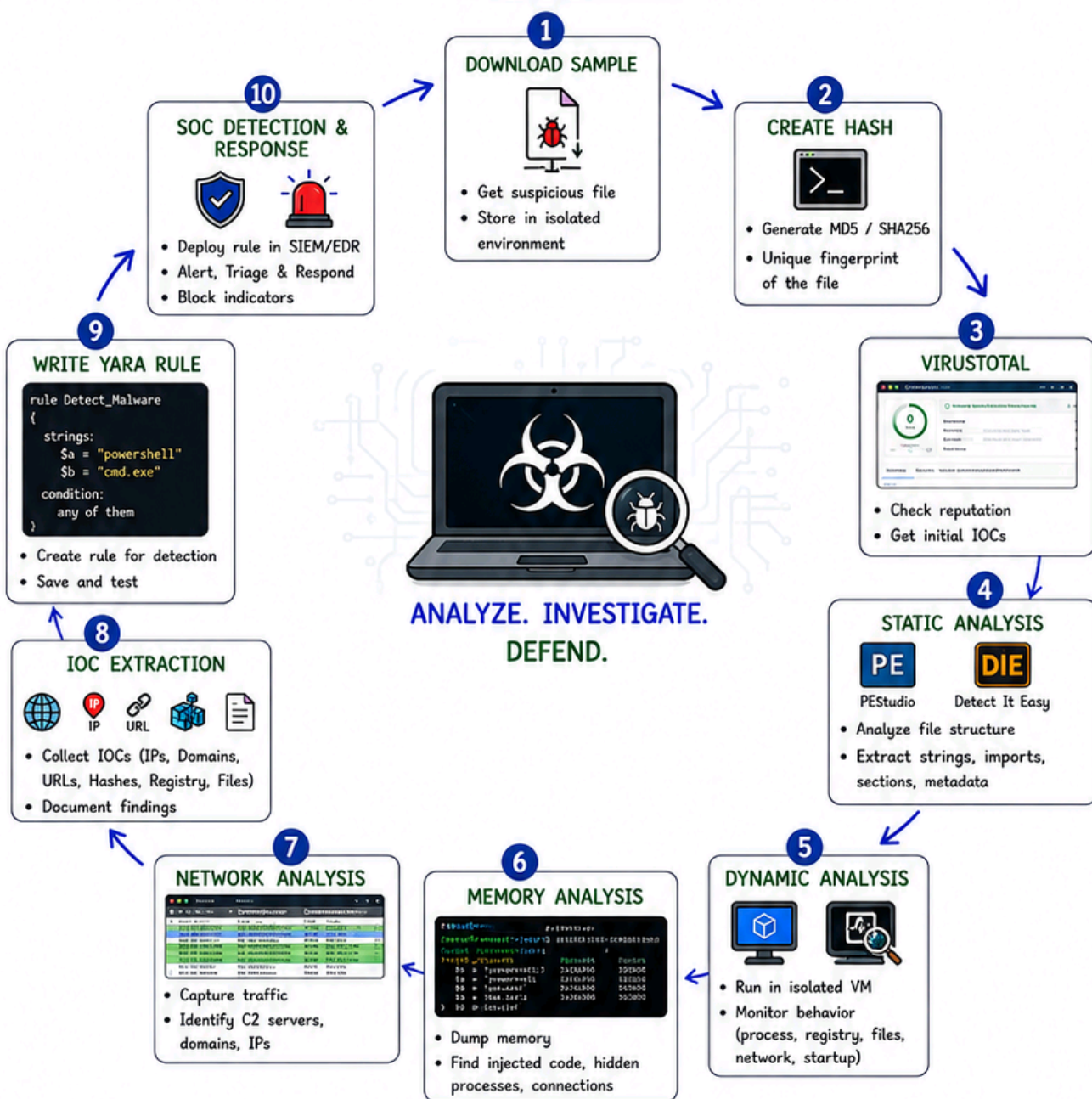
YARA helps SOC analysts quickly find malicious files using custom rules.





COMPLETE MALWARE ANALYSIS WORKFLOW

Follow this end-to-end workflow to analyze malware like a SOC Analyst using open-source tools.



KEY TAKEAWAY



Malware analysis is not just one step. It's a complete process. Every step gives you a piece of the puzzle. Put them together to see the full picture!

TOOLS USED

- ✓ Static Analysis : PEstudio, Detect It Easy, Strings
- ✓ Dynamic Analysis : Process Hacker, Procmon, Autoruns, Wireshark
- ✓ Memory Analysis : Volatility
- ✓ Network Analysis : Wireshark
- ✓ Detection : YARA
- ✓ Threat Intel : VirusTotal, AbuseIPDB, MISP

REMEMBER



- Always use isolated environment
- Never run malware on host machine
- Validate IOCs before blocking
- Document everything
- Share with your SOC team
- Continuous learning is the key!



GOAL: FOLLOW THE PROCESS, FIND THE TRUTH, STOP THE ATTACKER.

A strong process makes you a strong analyst! 💪





HANDS-ON EXERCISES & SOC INTERVIEW PREP

Practice these labs in your home lab and build real-world malware analysis skills step by step.

BEGINNER LABS

1 LAB 1 ANALYZE A PE FILE



- Calculate hash (MD5 / SHA256)
- Check file in VirusTotal
- Analyze with PESTudio
- Extract strings and imports

Tools: PESTudio, DIE, Strings

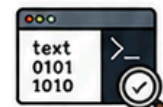
2 LAB 2 CAPTURE MALWARE TRAFFIC



- Run malware in isolated VM
- Capture traffic using Wireshark
- Identify domains, IPs, URLs
- Export suspicious packets

Tools: Wireshark, REMnux

3 LAB 3 EXTRACT STRINGS



- Extract ASCII / Unicode strings
- Identify sensitive information
- Find URLs, paths, commands
- Analyze string context

Tools: Strings, GREP, BinText

4 LAB 4 CREATE A YARA RULE



- Write a YARA rule
- Test rule on samples
- Improve rule accuracy
- Scan files and folders

Tools: YARA, VS Code

5 LAB 5 ANALYZE MEMORY



- Dump memory from VM
- Analyze with Volatility
- Find processes, injections, network connections

Tools: Volatility, FTK Imager

6 LAB 6 EXTRACT IOCs



- Extract all indicators
- Domains, IPs, Hashes, URLs
- Registry, Mutex, File paths
- Create IOC report

Tools: Manual, MISP, Excel

MALWARE ANALYSIS LAB CHECKLIST

- ☒ VMware / VirtualBox installed
- ☒ Windows 10 Analysis VM ready
- ☒ REMnux / Kali Linux installed
- ☒ Internet access for sandbox
- ☒ Wireshark working
- ☒ PESTudio installed and tested
- ☒ VirusTotal account created



- ☒ Static analysis completed
- ☒ Dynamic analysis completed
- ☒ Memory analysis completed
- ☒ Network analysis completed
- ☒ IOC report generated
- ☒ YARA rule created and tested
- ☒ Ready for SOC Analyst interviews!



SOC INTERVIEW PREP TOPICS



- Malware analysis methodology
- Common malware types & behavior
- Static vs Dynamic analysis
- IOC types and their importance
- YARA rules and detection
- Memory forensics basics
- Network traffic analysis
- Incident response workflow

BE A SOC ANALYST



- Think like an attacker
- Analyze like an investigator
- Document like a professional
- Respond like a defender

Practice. Learn. Defend.

That's the Blue Team mindset!



CONGRATULATIONS!

You have successfully built an **Open-Source Malware Analysis Home Lab** and completed a complete malware analysis workflow using free tools. You can now confidently begin practicing malware analysis and strengthen the practical skills expected from **SOC Analysts**, **DFIR analysts** and **Blue Team professionals**.

